Playwright notes

What is automation testing ?
Automation Testing is a software testing technique where test cases are executed automatically using tools and scripts to validate the functionality of an application without manual intervention.

What is playwright ?
Playwright is an open-source end-to-end automation testing framework developed by Microsoft that is used to automate web applications across multiple browsers.

What are the advantages of playwright ?
Playwright provides cross-browser support, built-in auto-waiting, faster execution, easy handling of multiple tabs, powerful locators, API testing, and network interception, making it a modern and stable automation tool.

What is async ?
`async` is used before a function to indicate that the function contains asynchronous operations.

What is await ?
`await` is used before asynchronous actions to make JavaScript wait until that action is completed.

What is async and await ?
`async` is used to declare an asynchronous function, and `await` is used to pause execution until a promise is resolved. In Playwright, they ensure browser actions complete before moving to the next step.

What are locators ?
Locators are used to identify and interact with web elements on a webpage. In automation testing, before performing any action like click, type, or verify, we must first locate the element. That identification method is called a locator.

What are CSS selectors ?
CSS Selectors are patterns used to locate and identify HTML elements on a webpage based on their attributes, structure, or position.
(or)
CSS selectors are used to locate web elements using id, class, tag name, or attributes in automation testing.

For ID we use #
Example
await page.locator('#username').fill('admin');

For Class we use .
Example

await page.locator('.login-btn').click();

For Tag Name we use tagName directly
Example
await page.locator('input').first().fill('text');

For attribute we use tagName[key:'value']
Example
await page.locator('[type="password"]').fill('1234');

Modern Locators in Playwright

1️⃣ getByRole()

Locates elements using ARIA role.

await page.getByRole('button', { name: 'Login' }).click();

👉 Most recommended locator
👉 Best for accessibility-based testing

2️⃣ getByText()

Locates elements using visible text.

await page.getByText('Submit').click();

3️⃣ getByLabel()

Locates input fields using associated label text.

await page.getByLabel('Username').fill('admin');

4️⃣ getByPlaceholder()

Locates elements using placeholder text.

await page.getByPlaceholder('Enter email').fill('test@gmail.com');

What are hooks in playwright ?

📌 Types of Hooks in Playwright

1️⃣ `beforeAll()`

Runs once before all tests in a file.

```
test.beforeAll(async () => {
 console.log("Runs once before all tests");
});
```

2️⃣ `beforeEach()`

Runs before every test.

```
test.beforeEach(async ({ page }) => {
 await page.goto('https://example.com');
});
```

3️⃣ `afterEach()`

Runs after every test.

```
test.afterEach(async () => {
 console.log("Runs after each test");
});
```

4️⃣ `afterAll()`

Runs once after all tests in a file.

```
test.afterAll(async () => {
 console.log("Runs once after all tests");
});
```

📌 Why Hooks Are Important?

- Avoid repeating code
- Maintain clean test structure

- Handle setup (like login)
- Handle cleanup (like closing connections)

Perfect 🔥
Now I'll give you **important Playwright interview questions with clear, placement-ready answers** (JavaScript focused).

I'll structure them short + professional so your students can answer confidently.

🎯 Playwright Interview Questions & Answers

1️⃣ What is Auto-Wait in Playwright?

✅ Answer:

Auto-wait is a built-in feature in Playwright where it automatically waits for elements to be visible, enabled, and stable before performing actions like click or fill.

It reduces flakiness and removes the need for manual waits in most cases.

2️⃣ What is the difference between `locator()` and `page.$()`?

✅ Answer:

`locator()` is recommended because:

- It supports auto-wait
- It retries automatically
- It is more stable

`page.$()` returns the element immediately and does not auto-wait, so it is less reliable.

3️⃣ What is Page Object Model (POM)?

✅ Answer:

Page Object Model is a design pattern where we create separate classes for each page and store locators and methods inside them to improve maintainability and reusability.

It helps in organizing large test projects.

4️⃣ How do you handle multiple tabs in Playwright?

✅ Answer:

We use `context.waitForEvent('page')` to capture the new tab.

Example:

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('#openTab')
]);
```

5️⃣ How do you handle frames in Playwright?

✅ Answer:

We use `frameLocator()`.

Example:

```
await page.frameLocator('#frameId').locator('#button').click();
```

6️⃣ How do you handle dropdowns?

✅ Answer:

For static dropdown:

```
await page.selectOption('#dropdown', 'value1');
```

For dynamic dropdown:
Use `getByText()` or locator with visible text after typing.

7️⃣ How do you take a screenshot?

✅ Answer:

```
await page.screenshot({ path: 'screenshot.png' });
```

For full page:

```
await page.screenshot({ path: 'full.png', fullPage: true });
```

8️⃣ What is `playwright.config.js`?

✅ Answer:

It is the configuration file where we define:

- Base URL
- Browser settings
- Parallel execution
- Timeout
- Reporter
- Headless mode

9️⃣ What is Parallel Execution?

✅ Answer:

Parallel execution means running multiple test files at the same time to reduce execution time.

Playwright supports this by default.

🔟 How do you run a specific test file?

✅ Answer:

npx playwright test filename.spec.js

1️⃣1️⃣ How do you run tests in headed mode?

✅ Answer:

npx playwright test --headed

1️⃣2️⃣ How do you handle alerts in Playwright?

✅ Answer:

```
page.on('dialog', async dialog => {
  await dialog.accept();
});
```

1️⃣3️⃣ What is Fixture in Playwright?

✅ Answer:

Fixtures are reusable components that provide test environment setup like browser, page, or custom data.

Playwright provides built-in fixtures like:

- page
- context
- browser

1️⃣4️⃣ How do you perform API testing in Playwright?

✅ Answer:

const response = await request.get('https://api.example.com/users');

expect(response.status()).toBe(200);

Playwright provides built-in APIRequestContext.

1️⃣5️⃣ What is the difference between Hard Wait and Explicit Wait?

✅ Answer:

Hard wait:

await page.waitForTimeout(5000);

(Not recommended)

Explicit wait:

await page.waitForSelector('#element');

(Recommended)

1️⃣6️⃣ How do you generate a report?

✅ Answer:

npx playwright show-report

Playwright provides HTML reports by default.

1️⃣7️⃣ What is Flaky Test?

✅ Answer:

A flaky test is a test that sometimes passes and sometimes fails without code changes, usually due to timing issues or improper waits.

1️⃣8️⃣ How is Playwright better than Selenium?

✅ Answer:

- Built-in auto wait
- Faster execution
- Modern architecture
- Better handling of multiple tabs
- Network interception support

2️⃣0️⃣ How do you retry failed tests?

✅ Answer:

In `playwright.config.js`:

retries: 2

🎯 Final Tip for Your Students

If they confidently answer:

- Auto wait
- Hooks
- POM
- Multiple tabs
- API testing
- Config file
- Parallel execution
- Tagging test
- Grouping tests
- Screenshots
- Alerts (dialog)
- Data driven test
- Handling pages (frames and iframes)
- Key board actions
- Mouse actions
- Visual tests
- Spc locators
- Css locators
- Trace
- Video
-

They can clear most Playwright interviews.
